Name: _____
Surname: _____
Date: _____

Instructions

Always assume each code is run independently, so variables do not persist between exercises.

Questions 1-6: Multiple Choice (5 points each)

On the table below, mark the correct answer for each question. Use an X to mark your choice. If you want to change your answer, fill the entire box and mark a new one. If multiple boxes are marked for the same question, no points will be awarded. See the examples below. Each question has **only one** correct answer. A correct answer awards 5 points. There is no penalty for wrong choices.

Example	A	B	C	D
Chosen A	X	☐	☐	☐
Chosen C, cancelled A	■	☐	X	☐
Chosen B, cancelled A and C	■	X	■	☐
Invalid (multiple choices)	X	X	☐	☐
Invalid (no choices)	■	☐	☐	☐

Questions 7-9: Debugging (10 points each)

These exercises show code cells with one or more errors. You have to find these errors, explain why they happen, and suggest a fix to them.

Questions 10-11: Open Problems (20 points each)

These are open-ended tasks. Explain your approach, outline algorithmic steps, and optionally provide compact working code snippets.

Mark your answers for Questions 1–6 below

Question	A	B	C	D
Q1	☐	☐	☐	☐
Q2	☐	☐	☐	☐
Q3	☐	☐	☐	☐
Q4	☐	☐	☐	☐
Q5	☐	☐	☐	☐
Q6	☐	☐	☐	☐

1. Which of these statements is **FALSE** about **for** loops?

   A. A for loop iterates over any iterable.

   B. A for loop can use two loop variables. (e.g. for i, value in iterable)

   C. After a for loop finishes, the loop variable keeps its last assigned value.

   D. A for loop only iterates over immutable iterables.

2. After running the following code, what will be the final value of variable "str1"?

```
str1 = "GOOD LUCK"
for index in range(len(str1)):
  str1[index] = str1[index].lower()
```

   A. `str1 = "GOOD LUCK"`

   B. `str1 = "good luck"`

   C. The code will raise an error.

   D. `str1 = "GOOD LUCK good luck"`

3. Which of these statements is **FALSE** about functions?

   A. You can access global variables inside a function.

   B. We can directly modify a list inside a function without needing to use a return statement.

   C. A function must always have return statement.

   D. The print() statement inside a function only displays a value, but it does not return any output.

4. What will be the output value of the following code?

```
shopping_list = ["Bread", "Eggs", "Banana", "Cheese", "Olive oil"]

fruit_index = shopping_list.index("Banana")
# Outputs the index of "Banana" in the list

shopping_list[fruit_index + 1:fruit_index + 1] = ["Orange", "Apple"]
print(shopping_list)
```

   A. `["Bread", "Eggs", "Banana", "Orange", "Apple", "Cheese", "Olive oil"]`

   B. The code will raise an error.

   C. `["Bread", "Eggs", "Orange", "Apple", "Banana", "Cheese", "Olive oil"]`

   D. `["Bread", "Eggs", "Banana", "Cheese", "Olive oil", "Orange", "Apple"]`

5. After running this code to find the first five even numbers, what will be the final value of the variable "count"?

```
count = 0
number = 0

while count < 5:
  if number%2 == 0:
    count += 1
    print(number)
  else:
    count = 0
  number += 1
```

   A. count = 4

   B. The code will never stop running.

   C. count = 0.

   D. count = 5

6. After running this code, what will be the final value of `serie_set`?

```
def difference_add_serie(number_list: list[int]):
    for i in range(1, len(number_list)):
        yield number_list[i] - number_list[i - 1]
        yield number_list[i] + number_list[i - 1]

number_list = [1, 2, 3, 6, 7]
serie_set = set()
for result in difference_add_serie(number_list):
    serie_set.add(result)
```

   A. $\{1, 3\}$

   B. $\{3, 5, 9, 13\}$

   C. $\{1, 3, 5, 9, 13\}$

   D. $\{1, 3, 1, 5, 3, 9, 1, 13\}$

Correct answers for Questions 1–6:

Question	Key
1	D
2	C
3	C
4	A
5	B
6	C

7. The function below is designed to remove all vowels from a given string. However, it doesn't remove all the vowels as expected. Identify the problem and fix the code.

```
def remove_vowels(input_str: str) -> str:
        vowels = ("a", "e", "i", "o", "u")
        result = ""
        for char in input_str:
                if char.lower() in vowels:
                        input_str.replace(char, "")
                return input_str

text = "Programming"
print(remove_vowels(text))  # Output: "Programming"
```

Answer:

The first problem is that strings are immutable: the method `input_str.replace(char, "")` does not modify `input_str`, it returns a new string. As we do not store that new string anywhere, nothing is removed. The variable `result` is created but never used. *(+4 points)*

The second problem is that `return input_str` is indented inside the `for` loop, so the function ends after checking only the first character. The `return` must be placed after the loop. *(+3 points)*

To fix it, we use `result` to build the output: for every character, if it is not a vowel, we add it to `result` with `result += char`. After the loop, we return `result`. *(+3 points)*

(Optional, fixed code)

```
def remove_vowels(input_str: str) -> str:
    vowels = ("a", "e", "i", "o", "u")
    result = ""
    for char in input_str:
        if char.lower() not in vowels:
            result += char
    return result

text = "Programming"
print(remove_vowels(text))  # Prgrmmng
```

8. You have created a function that stacks two 1D-arrays, one of top of another. If the arrays have a different length, the function fills that difference with zeros. Your function raises an error on line 6: "`ValueError: could not broadcast input array from shape (3,) into shape (0,)`". Find the mistake(s) and correct it.

```
1  import numpy as np
2
3  def stack_uneven(x: np.array, y: np.array) -> np.array:
4          if len(x) > len(y):
5                  y = np.zeros(len(x))
6                  y[len(y):] = y
7          else:
8                  x = np.zeros(len(y))
9                  x[len(x):] = x
10         return np.stack((x, y))
11
12 x = np.array([1, 2, 3])
13 y = np.array([4, 5])
14 print(stack_uneven(x, y))
15 # Expected:
16 # [[1, 2, 3],
17 #  [4, 5, 0]]
18 # Output:
19 # [[1., 2., 3.],
20 #  [0., 0., 0.]]
```

Answer:

In the `if` branch, the line `y = np.zeros(len(x))` overwrites `y` with an array of zeros, so we lose the original values of `y`. Then, `y[len(y):] = y` tries to assign three values to an empty slice (the slice starts at the end of the array), which is the `ValueError` we see. *(+4 points)*

We have to create the zero-padded array under a different name, and copy the original values into its first positions, using the **original** length: `padded[:len(y)] = y`. Then we can use the padded array in place of `y`. *(+4 points)*

The `else` branch has exactly the same mistake with `x`, so it has to be fixed in the same way. *(+2 points)*

Comment: `np.zeros()` creates floats by default, so the result would be `[4., 5., 0.]` instead of `[4, 5, 0]`. We can keep the integers by using `dtype=y.dtype`.

(Optional, fixed code)

```
1  import numpy as np
2
3  def stack_uneven(x: np.array, y: np.array) -> np.array:
4      if len(x) > len(y):
5          padded = np.zeros(len(x), dtype=y.dtype)
6          padded[:len(y)] = y
7          y = padded
8      else:
9          padded = np.zeros(len(y), dtype=x.dtype)
10         padded[:len(x)] = x
11         x = padded
12     return np.stack((x, y))
```

```
13
14 x = np.array([1, 2, 3])
15 y = np.array([4, 5])
16 print(stack_uneven(x, y))
17 # [[1 2 3]
18 #  [4 5 0]]
```

9. The following code should join three arrays together to form the first nine letters of the alphabet. When you run it, the last line returns an error: "`AttributeError: "numpy.ndarray" object has no attribute "concatenate"`". Explain the issue(s) and fix it.

```
1 import numpy as np
2
3 nm = np.array([["a", "b", "c"]])
4 nn = np.array(["d", "e", "f"])
5 np = np.array(["g", "h", "i"])
6
7 nq = np.concatenate([nm, nn, np])
```

Answer:

The first problem is on line 6: `np = np.array(["g", "h", "i"])` overwrites the name `np`, which was our alias of the NumPy library, with an array. Then, `np.concatenate` is looking for that method inside an array, which does not exist. We have to use a different variable name, for instance `no`, and use it also in the concatenation. *(+5 points)*

Once this is fixed, a second error appears: `nm` is defined with double square brackets, so it is a 2D array of shape `(1, 3)`, while `nn` and the third array are 1D arrays of shape `(3,)`. `np.concatenate` needs all the arrays to have the same number of dimensions, so it raises a `ValueError`. We must define `nm` with single square brackets. *(+5 points)*

(Optional, fixed code)

```
1 import numpy as np
2
3 nm = np.array(["a", "b", "c"])
4 nn = np.array(["d", "e", "f"])
5 no = np.array(["g", "h", "i"])
6
7 nq = np.concatenate([nm, nn, no])
8 print(nq)  # ['a' 'b' 'c' 'd' 'e' 'f' 'g' 'h' 'i']
```

10. You are given a list of reports, each report consisting of a string of single digits separated by spaces, for instance `"1 2 3"`. A report is considered valid if it meets the following conditions:

    1. All digits are strictly increasing from start to end.
    2. The absolute difference between any two adjacent digits is at least 1 and at most 3.

    Write a function `count_valid_reports()` that, given **any** list of reports of **any size**, returns an integer: the number of valid reports. For instance:

```
    ls_reports = [
            "1 2 7 8 9",
            "1 3 2 4 5",
            "1 4 4 6 8",
            "1 3 6 7 9"
    ]

    print(count_valid_reports(ls_reports))  # Output: 1
```

Answer:

As we need to check several reports inside a list, the best approach is to use a for loop. We look at one report at a time and apply the same rules to each of them: `for report in ls_reports`. *(+5 points)*

Because we want to count the valid reports, before the loop starts we initialize a counter, `count = 0`. Inside the loop, every valid report increases it with `count += 1`. At the end of the function, after the loop finishes, we return the resulting count. *(+5 points)*

Inside the loop we have a single report, which is a string. We split it by its spaces with `report.split(" ")`, which returns a list of strings, and turn every string into a number with a list comprehension and `int()`: `digits = [int(s) for s in report.split(" ")]`. For instance, `"1 3 6"` becomes `[1, 3, 6]`. *(+5 points)*

Now we start a nested loop, `for idx in range(1, len(digits))`, to compare every digit against the previous one, with `diff = digits[idx] - digits[idx-1]`. Both rules are met if and only if `1 <= diff <= 3`: a difference of at least 1 means strictly increasing, and at most 3 is the maximum allowed jump. Before the nested loop we set `is_valid = True`. If the difference is outside that range, we set `is_valid = False` and `break`, as a single failure is enough. After the nested loop, if `is_valid` is still `True`, we increase the count. *(+5 points)*

Comment: Checking only that the digits increase is not enough, as `"1 2 7 8 9"` is increasing but has a jump of 5. An empty list of reports returns 0, and a report with a single digit has no pairs to fail, so it counts as valid.

(Optional, code)

```
def count_valid_reports(ls_reports: list[str]) -> int:
    count = 0
    for report in ls_reports:
        # Example: "1 3 6" -> [1, 3, 6]
        digits = [int(s) for s in report.split(" ")]
        is_valid = True
        for idx in range(1, len(digits)):
            diff = digits[idx] - digits[idx - 1]
```

```
            if diff < 1 or diff > 3:
                is_valid = False
                break
        if is_valid:
            count += 1
    return count
```

11. Your task is to simulate the circular motion of an object by computing its coordinates. For a circle with radius R and an object moving along its circumference, you need to determine the object's x and y positions corresponding to a list of angles α (in degrees).

    The coordinates are determined by the following formulas:

$$x = R \cdot \cos(\alpha)$$
$$y = R \cdot \sin(\alpha)$$

    Part 1: Implement the function `circular_motion(radius, angles)`:

    - `radius`: A scalar value denoting the radius of the circle, R.
    - `angles`: A list of angles in **degrees**, α.

    The function should compute and return two NumPy arrays: `x_coords` and `y_coords`. These arrays represent the x and y coordinates on the circle for each angle in `angles`. Round this values to two decimals.

    Part 2: Implement a function `verify_coordinates(radius, x_coords, y_coords)` to verify the accuracy of your circular motion function. For each coordinate pair (x, y) in `x_coords` and `y_coords`, check if the equation $\sqrt{x^2 + y^2}$ equals the radius R within a reasonable tolerance (e.g., 0.01). This validation ensures that each calculated point lies on the circle. The function should return "True" if all points are on the circle and "False" otherwise.

Answer:

Part 1. The function receives the radius and a list of angles in degrees, and returns two arrays. The trigonometric functions of NumPy work in **radians**, so we must first convert the angles, using `np.radians(angles)`. This function also accepts a list, so we do not need a loop. *(+4 points)*

Then we apply the formulas to the whole array at once, using vectorization: `x = radius * np.cos(rad)` and `y = radius * np.sin(rad)`. Each element is computed independently, so every angle gives its own coordinates. *(+3 points)*

We round both arrays to two decimals with `np.round(..., 2)` and return them as two separate arrays. *(+3 points)*

Part 2. The function receives the radius and the two arrays of coordinates. For each point, the distance to the center is `np.sqrt(x**2 + y**2)`, which we compute for all the points at once. *(+4 points)*

We cannot compare this distance with the radius using `==`, because the coordinates were rounded to two decimals, so the distances are only approximately equal. Instead, we compare the absolute difference against the tolerance, `np.abs(distance - radius) <= 0.01`, which gives a boolean array. *(+3 points)*

Finally, we use `np.all()` to check that every point passes the test, and return that single boolean. *(+3 points)*

(Optional, code)

```python
import numpy as np

def circular_motion(radius, angles):
    # NumPy trigonometric functions use radians
    rad = np.radians(angles)
    x_coords = np.round(radius * np.cos(rad), 2)
    y_coords = np.round(radius * np.sin(rad), 2)
    return x_coords, y_coords

def verify_coordinates(radius, x_coords, y_coords):
    distance = np.sqrt(x_coords**2 + y_coords**2)
    on_circle = np.abs(distance - radius) <= 0.01
    return bool(np.all(on_circle))

x, y = circular_motion(5, [0, 45, 90, 180])
print(verify_coordinates(5, x, y))  # True
```

Cheatsheet

Given a number named `x`:

- `x % y` returns the remainder of the division $\frac{x}{y}$.
- `x // y` returns the quotient of the division $\frac{x}{y}$.
- `abs(x)` returns the absolute value of `x`.

Given a string named `s`:

- `int(s)` returns an integer. Fails if `s` has invalid characters.
- `float(s)` returns a floating number. Fails if `s` has invalid characters.
- `s.lower()` returns a copy of the string, all in lowercase.
- `s.split(" ")` split a string into a list where each word is a list item.
- `s.count(<pattern>)` returns the number of non-overlapping occurrences of `<pattern>`.
- `s.find(<pattern>, <start>)` finds the first occurrence of the specified `<pattern>`, starting from index `<start>`. Returns `-1` if the pattern is not found.
- `s.replace(<pattern>, <new>)` returns a new string where all instances of `<pattern>` have been replaced with `<new>`.

Given a list named `ls`:

- `ls.append(element)` add element to that list.

Given a list or tuple named `ls`:

- `ls.index(element)` returns the index of element in that list.
- `len(ls)` returns the number of elements in that list.
- `sum(ls)` returns the sum of the elements of that list.
- `sorted(ls)` returns a sorted version of that list.
- `tuple(ls)` / `set(ls)` returns a tuple / set using the elements of that list.
- `", ".join(ls)` returns a string with the elements of the list separated by commas.

Given a set named `a`:

- `a.add(x)` adds the element `x` to the set `a`.
- `a.union(b)` or (`a | b`) returns the union of set `a` with another set `b`.
- `a.intersection(b)` or (`a & b`) returns the intersection of set `a` with another set `b`.
- `a.difference(b)` or (`a - b`) returns the difference of set `a` with another set `b`.

Given a dictionary named `d`:

- `list(d.keys())` returns a list with the keys of `d`.
- `list(d.values())` returns a list with the values of `d`.
- `list(d.items())` returns a list with the tuples (key, value) of `d`.

Some useful build-in functions:

- `type(x)` returns the type of `x` (int, float, list...).
- `range(start, stop)` generates numbers from `start` until `stop`, not including `stop`.
- `input(msg)` is used to take user input, in form of a string.

NumPy Cheatsheet

Importing NumPy:

- `import numpy as np`
- `rng = np.random.default_rng()`

Creating arrays:

- `np.zeros((m, n))` creates an $m \times n$ array filled with zeros.
- `np.ones((m, n))` creates an $m \times n$ array filled with ones.
- `np.arange(start, stop, step)` creates values from `start` to `stop` (exclusive) with a given `step`.
- `np.linspace(start, stop, num)` creates `num` evenly spaced values between `start` and `stop`.

Given a NumPy array `arr`:

- `arr.ndim` returns the number of array dimensions, as a number.
- `arr.shape` returns the shape of an array, as a tuple.

Array operations:

- `np.dot(a, b)` computes the dot product.
- `np.matmul(a, b)` or `a @ b` performs matrix multiplication.
- `np.transpose(a)` or `a.T` transposes the array.

Array manipulation:

- `a.reshape((m, n))` changes the shape of an array.
- `a.flatten()` flattens a multidimensional array into 1D.
- `np.concatenate((a, b), axis=0)` joins arrays along an axis.

- `np.stack(<sequence>)` joins a sequence of arrays along a new axis.
- `np.vstack((a, b))`, `np.hstack((a, b))` vertical/horizontal stacking.
- `np.split(a, n)` splits an array into `n` parts.

Statistics and descriptive analysis:

- `np.mean(a)` mean of all elements.
- `np.median(a)` median of all elements.
- `np.std(a)` standard deviation.
- `np.var(a)` variance.
- `np.min(a)`, `np.max(a)` minimum and maximum.
- `np.argmin(a)`, `np.argmax(a)` indices of minimum and maximum.
- `np.corrcoef(a, b)` correlation coefficient matrix.
- `np.cov(a, b)` covariance matrix.

Linear algebra:

- `np.linalg.inv(a)` inverse of matrix `a`.
- `np.linalg.det(a)` determinant of `a`.
- `np.linalg.eig(a)` eigenvalues and eigenvectors.
- `np.linalg.norm(a)` vector or matrix norm.
- `np.linalg.solve(a, b)` solves $Ax = b$.

Logical operations and conditions:

- `np.where(cond, x, y)` selects elements from `x` or `y` based on condition.
- `np.all(a)`, `np.any(a)` check if all/any elements are True.
- `np.unique(a)` returns sorted unique elements.

Random number generation (using `rng`):

- `rng.random(size)` uniform distribution in $[0, 1)$.
- `rng.integers(low, high, size)` random integers in $[low, high)$.
- `rng.normal(mean, std, size)` normal (Gaussian) distribution.
- `rng.uniform(low, high, size)` uniform distribution in $[low, high)$.
- `rng.choice(a, size)` random sample from array `a`.

Random number generation (legacy API):

- `np.random.randint(low, high, size)` returns random integers from low (inclusive) to high (exclusive).
- `np.random.uniform(low, high, size)` samples from a uniform distribution.
- `np.random.seed(seed)` fixes the seed. After a seed reset, the same set of numbers will appear every time.

Math and trigonometry:

- `np.power(arr1, arr2)` returns the first array elements raised to powers from second array, element-wise.
- `np.sin(arr)` returns the trigonometric sine, assuming angle in radians.
- `np.cos(arr)` returns the trigonometric cosine, assuming angle in radians.
- `np.radians(arr)` converts angles from degrees to radians.
- `np.degrees(arr)` converts angles from radians to degrees.